

PROYECTO DE CURSO

Documento de Apoyo 04: (Semana 10)

750021C - DESARROLLO DE SOFTWARE II



FACULTAD DE INGENIERÍA
Escuela de Ingeniería de Sistemas y
Computación

FEBRERO - JUNIO / 2026

Webhooks y Validación de Pull Requests

1. Objetivos

- 1.1. Configurar la comunicación en tiempo real entre GitHub y Jenkins mediante Webhooks.
- 1.2. Implementar la validación automática de Pull Requests para proteger la rama de integración.
- 1.3. Automatizar el reporte de estado de Jenkins hacia la interfaz de GitHub.

2. Información Clave

- **Webhooks:** Notificaciones automáticas que GitHub envía a Jenkins cuando ocurre un evento.
- **Payload URL:** La dirección pública del Jenkins en el VPS que recibirá la notificación.
- **Status Checks:** Los indicadores que aparecen en GitHub para informar si el pipeline pasó o falló.
- **Trigger de PR:** Configuración para que Jenkins evalúe el código antes de ser mezclado en `develop`.

3. Procedimientos

A. Configuración del Webhook en Github

1. Ingresar al repositorio en GitHub.
2. Ir a **Settings** > **Webhooks** > **Add webhook**.
3. **Payload URL**:
`http://45.55.145.98:900X/github-webhook/`
(Es vital incluir el último /).
4. **Content type**: `application/json`.
5. **Eventos**: Seleccionar **Let me select individual events** y marcar:
 - a. **Pushes**
 - b. **Pull requests**
6. Guardar y verificar que aparezca un  de conexión exitosa.

B. Ajuste del Job en Jenkins

1. Ingresar a su instancia de Jenkins.
2. En la configuración del Job de Pipeline creado con el **Documento de Apoyo 03**:
 - Desactivar el **Polling SCM** (ya no es necesario preguntar, ahora GitHub avisará).
 - Asegurar que **GitHub hook trigger for GITScm polling** esté marcado.
3. En la sección **Pipeline**, asegurar que el **Branch Specifier** permita procesar ramas de PR, usualmente configurado como `*/develop` o dejando que el plugin de GitHub maneje las referencias.

C. Modificación del Jenkinsfile para Reporte de Estado

Es recomendable añadir un bloque `post` para recibir feedback:

Jenkinsfile

```
pipeline {
    agent any

    stages {
        // ... etapas anteriores ...
        // ... no es necesario modificar ...
    }

    post {
        success {
            echo '¡El Pipeline pasó! El código es seguro para
integrar.'
        }
    }
}
```

```

failure {
    echo 'El Pipeline falló. Revisar logs antes de
reintentar el PR.'
}
}
}

```

4. Resultados Esperados

- Al crear un Pull Request desde una rama **feature/** hacia **develop**, Jenkins debe dispararse inmediatamente.
- En la pestaña **Conversation** del PR en GitHub, debe aparecer el estado del build (amarillo mientras se ejecuta, verde si pasa, rojo si falla).
- El equipo debe ser capaz de bloquear un PR si Jenkins reporta un fallo, cumpliendo con el *Gatekeeping* de calidad.

5. Solución de Problemas

Problema	Posible Causa	Solución Sugerida
El Webhook da error 404	La URL del Payload está mal escrita o falta el / final.	Revisar que la URL termine exactamente en /github-webhook/ .
El Webhook da error 500 o Timeout	El Jenkins en el VPS no está respondiendo o el puerto está bloqueado.	Verificar acceso a la interfaz web de Jenkins; si carga, revisar el log de Webhooks en GitHub para ver el detalle del error.
Jenkins no reporta el estado a GitHub	El PAT (Token) no tiene permisos de repo:status .	Generar un nuevo PAT en GitHub asegurando que la opción de Commit statuses esté marcada y actualizar la credencial en Jenkins.